Tarea Complementaria

Problema 1. Bagging, Random Forest y varianza del ensamble

Un hospital universitario desea mejorar la detección automática de arritmias cardíacas a partir de características extraídas de señales ECG. Para ello, se plantea el uso de métodos de ensamble basados en árboles de decisión. Antes de implementar el sistema, el equipo de ingeniería biomédica necesita entender los fundamentos del bagging**,** su relación con Random Forest y su efecto sobre la varianza del modelo.

Se pide:

a) Explique cuál es la idea central del bagging. ¿En qué se diferencia conceptualmente del boosting? Mencione al menos dos diferencias en términos de entrenamiento, objetivo y riesgo principal de cada método.

El bagging (Bootstrap Aggregating) es una técnica de ensamble cuyo objetivo principal es reducir la varianza de modelos inestables, como los árboles de decisión. La idea central consiste en generar múltiples subconjuntos de entrenamiento a partir del conjunto de datos original mediante muestreo con reemplazo (bootstrap). Sobre cada uno de estos subconjuntos se entrena un modelo independiente, y posteriormente sus predicciones se combinan mediante votación (en clasificación) o promedio (en regresión). Este proceso permite que los errores individuales de cada modelo, al ser diferentes entre sí, se compensen al agregarse, logrando así un modelo final más robusto y con menor variabilidad frente a cambios en los datos. En contraste, el boosting es otra técnica de ensamble que sigue un enfoque distinto. Mientras que el bagging entrena los modelos de forma independiente y en paralelo, el boosting construye los modelos de manera secuencial, donde cada nuevo modelo se entrena poniendo mayor énfasis en los errores cometidos por los modelos anteriores. De esta manera, el boosting busca mejorar progresivamente el desempeño del modelo, corrigiendo los errores acumulados en cada iteración.

En términos de objetivo, el bagging se enfoca principalmente en reducir la varianza del modelo, lo que lo hace especialmente útil cuando se trabaja con modelos que tienden a sobre ajustarse. Por otro lado, el boosting busca reducir tanto el sesgo como la varianza, construyendo un modelo fuerte a partir de la combinación de múltiples modelos débiles. Sin embargo, esta diferencia en el enfoque también implica distintos riesgos: el bagging puede ser menos efectivo si los modelos generados son muy similares entre sí, mientras que el boosting es más propenso al sobreajuste, especialmente en presencia de ruido o valores atípicos, debido a su énfasis en corregir errores difíciles.

En resumen, aunque ambas técnicas pertenecen a la familia de métodos de ensamble, el bagging se basa en la independencia y la agregación de modelos para reducir la varianza, mientras que el boosting utiliza una estrategia secuencial orientada a mejorar iterativamente el desempeño, lo que le permite reducir tanto sesgo como varianza, pero con un mayor riesgo de sobreajuste.

b) Considere un ensamble formado por $m$predictores entrenados sobre muestras bootstrap.

Suponga que cada predictor tiene varianza individual σ^2 y que la correlación promedio entre predictores es ρ. La varianza del predictor ensamblado puede expresarse como:

$$Var(\bar{y}) = \left(\frac{1-\rho}{m} + \rho\right)\sigma^2$$

A partir de esta expresión, responda:

- ¿El bagging reduce principalmente el sesgo, la varianza o ambos? Justifique.

  El bagging reduce principalmente la varianza del modelo. Esto se observa en la expresión de la varianza del ensamble, donde el término que depende del número de modelos mmm permite disminuir la variabilidad de las predicciones al promediarlas. Sin embargo, el sesgo se mantiene prácticamente igual, ya que todos los modelos base tienen la misma estructura (por ejemplo, árboles de decisión) y no se modifica su capacidad de ajuste. Por ello, el bagging es especialmente útil para modelos con alta varianza, pero no corrige el sobreajuste.
- ¿Qué ocurre con $Var(y^{-})$ cuando m aumenta?

  Cuando el número de predictores m aumenta, el término $\frac{1-p}{m}$ disminuye y tiende a cero. Como resultado, la varianza del ensamble se aproxima a $\rho\sigma^2$. Lo que implica que al agregar más modelos, la varianza disminuye, pero solo hasta un límite, por lo que el beneficio de aumentar m presenta rendimiento decrecientes.
- ¿Qué ocurre si ρ es muy alto? Interprete el resultado.

  Si la correlación promedio ρ es muy alta (cercana a 1), la varianza del ensamble se aproxima a σ^2, que es la varianza de un modelo individual. Esto significa que el ensamble no logra reducir la varianza, ya que todos los modelos cometen errores similares. En consecuencia, la efectividad del bagging depende de que los modelos sean lo suficientemente diversos (baja correlación).

- En Random Forest se seleccionan aleatoriamente algunas features en cada nodo. Explique cómo esto afecta a ρ y a la varianza del ensamble.

  La selección aleatoria de características en cada nodo reduce la correlación ρ\rhoρ entre los árboles, ya que cada uno se construye utilizando diferentes subconjuntos de variables y, por lo tanto, aprende patrones distintos. Al disminuir ρ\rhoρ, se reduce el término dominante en la expresión de la varianza del ensamble, lo que permite obtener una mayor reducción de la varianza total. Por ello, Random Forest mejora el desempeño del bagging al fomentar la diversidad entre los modelos.

c) Defina el error out-of-bag (OOB) y explique por qué puede utilizarse como una estimación del error de generalización sin separar explícitamente un conjunto de validación.

El error out-of-bag (OOB) es una estimación del error de generalización en métodos como bagging y Random Forest. Se basa en que, al crear cada muestra bootstrap, aproximadamente el 37% de los datos no se utilizan para entrenar un árbol. Estas muestras se denominan out-of-bag. Para calcularlo, cada observación se predice usando solo los modelos en los que no fue utilizada durante el entrenamiento. Luego, se comparan esas predicciones con los valores reales y se promedia el error.

El OOB funciona como una estimación del error de generalización porque evalúa el modelo sobre datos no vistos, sin necesidad de separar un conjunto de validación. Además, permite aprovechar todo el dataset de forma eficiente.

d) En un bootstrap de tamaño n, una muestra específica tiene probabilidad aproximada de no ser seleccionada:

$$\left(1 - \frac{1}{n}\right)^n \approx e^{-1} \approx 0.368$$

Explique con sus propias palabras qué significa este resultado y cómo se relaciona con el cálculo del error OOB.

Este resultado básicamente dice que cuando hacemos un bootstrap de tamaño n, cada dato tiene como un 36.8% de probabilidad de no ser elegido. O sea, siempre hay una parte de los datos que queda “afuera” en cada árbol. Esto es clave para el error OOB, porque esos datos que no se usaron en el entrenamiento de un modelo se pueden usar como si fueran datos de prueba. Entonces, para cada muestra, el modelo se evalúa con árboles que nunca la vieron. En otras palabras, gracias a esto no necesitamos separar un set de validación aparte, porque el mismo proceso de bootstrap ya nos deja datos “nuevos” para evaluar. Por eso el OOB es una buena aproximación del error real del modelo.

Problema 2. Árbol de decisión, índice Gini y profundidad máxima

Se dispone del siguiente dataset biomédico de 15 pacientes hospitalizados, con tres características extraídas de su evaluación cardiológica:

Paciente	x_1	x_2	x_3	Clase y
A	5.5	0.5	4.5	2
B	7.4	1.1	3.6	0
C	5.9	0.2	3.4	2
D	9.9	0.1	0.8	0
E	6.9	-0.1	0.6	2
F	6.8	-0.3	5.1	2
G	4.1	0.3	5.1	1
H	1.3	-0.2	1.8	1
I	4.5	0.4	2.0	0
J	0.5	0.0	2.3	1
K	5.9	-0.1	4.4	0
L	9.3	-0.2	3.2	0
M	1.0	0.1	2.8	1
N	0.4	0.1	4.3	1
O	2.7	-0.5	4.2	1

La variable objetivo y indica el diagnóstico al alta:

- 0: sin evento cardíaco adverso
- 1: riesgo intermedio documentado
- 2: evento cardíaco adverso

confirmado Se pide:

- Construya un árbol de decisión con profundidad máxima igual a 2. Evalúe los posibles splits en las variables x_1, x_2y x_3, utilizando el índice de Gini como criterio de división:

$$Gini = 1 - \sum_{k=1}^{K} p_k^2$$

Para cada split relevante, calcule la reducción de impureza ponderada:

$$\Delta Gini = Gini_{padre} - \left(\frac{n_L}{n} Gini_L + \frac{n_R}{n} Gini_R\right)$$

Seleccione el split que maximiza $\Delta Gini$ en cada nodo.

Separaremos la clase y

Clase 0: 5 pacientes
Clase 1: 6 pacientes
Clase 2: 4 pacientes

Y calculamos el Gini padre

Gini padre = 1 - $[(\frac{5}{15})^2 + (\frac{6}{15})^2 + (\frac{4}{15})^2]$
Gini padre = 0.658
Esto nos indica que el nodo inicial tiene una impureza considerable ya que tiene pacientes de 3 clases

Ahora evaluaremos los splits y evaluaremos los distintos thresholds entre valores consecutivos de x1, x2 y x3 evitando divisiones que generen nodos con menos de 2 muestras.

El mejor split encontrado fue:

x1 <= 4.3 este split genera la mayor reducción de impureza
$\Delta Gini = 0.361$
Si x1 <= 4.3 serian 6 pacientes todos en clase 1
Si x1 > 4.3 serian 9 pacientes con clases 0 y 2

Como el árbol tiene profundidad máxima igual a 2, solo se puede hacer una división adicional en los nodos hijos. El nodo izquierdo ya es puro, por lo que no necesita dividirse más.

En el nodo derecho, donde x1>4.3, se vuelve a evaluar el índice Gini. El mejor split fue:

x1≤7.15

Este split separa los 9 pacientes restantes en:

Si x1≤7.15: mezcla de clase 0 y clase 2.

Si x1>7.15: 3 pacientes, todos de clase 0.

El árbol de decisión con profundidad máxima 2 queda:

Si x1≤4.3 corresponde a Clase 1
Si x1>4.3 y x1≤7.15 corresponde a Clase 2
Si x1>7.15 corresponde a Clase 0

El árbol seleccionó primero la variable x1, porque fue la característica que produjo la mayor reducción de impureza. Esto significa que x1 tiene una buena capacidad para separar a los pacientes según su diagnóstico. Además, el primer split identifica claramente a los pacientes de

clase 1 cuando x1≤4.3. Luego, para los pacientes restantes, una nueva división en x1 permite separar mejor las clases 0 y 2.

- Para cada hoja final del árbol, indique:
    - número de muestras,
    - distribución de clases,
    - clase predicha,
    - valor aproximado del índice Gini.

Hoja 1: x1≤4.3

- Número de muestras: 6
- Distribución de clases:
    - Clase 0: 0
    - Clase 1: 6
    - Clase 2: 0
- Clase predicha: Clase 1
- Índice Gini:

Gini= $1 - (1)^2 = 0$

Nodo completamente puro.

Hoja 2: 4.3<x1≤7.15

- Número de muestras: 6
- Distribución de clases:
    - Clase 0: 2
    - Clase 1: 0
    - Clase 2: 4
- Clase predicha: Clase 2 (mayoría)
- Índice Gini:

Gini = $1 - [(\frac{2}{15})^2 + (\frac{4}{15})^2] = 0.44$

Nodo impuro pero dominado por clase 2

Hoja 3: x1>7.15

- Número de muestras: 3
- Distribución de clases:
    - Clase 0: 3
    - Clase 1: 0

    - Clase 2: 0
- Clase predicha: Clase 0
- Índice Gini:

Gini= 1 - $(1)^2 = 0$

Nodo completamente puro

Se observa que dos de las tres hojas son completamente puras (Gini = 0), lo que indica que el árbol logra separar correctamente ciertos grupos de pacientes. La única hoja impura corresponde a una región donde aún existe solapamiento entre clases 0 y 2, lo cual es esperable dado que la profundidad del árbol está limitada a 2.

c) Utilice el árbol obtenido para clasificar los siguientes nuevos pacientes:

$$x_a = (4.1, -0.1, 2.2)^T$$

$$x_b = (6.1, 0.4, 1.3)^T$$

Para cada vector, proporcione:

- clase predicha,
- hoja terminal alcanzada,
- probabilidad empírica $p(\ c \mid x, T\ )$estimada a partir de la distribución de clases en la hoja.
- Explique brevemente por qué limitar la profundidad del árbol puede ayudar a evitar el sobreajuste en problemas biomédicos.

Hacemos que x1 = 4.1

4.1 <= 4.3
Hoja terminal alcanzada: Hoja 1
Distribución de clases en la hoja: [0,6,0]
Clase predicha: Clase 1
Probabilidad empírica:

p(c=1|xa,T)=6/6=1

Por lo tanto el paciente xa se clasifica como riesgo intermedio documentado

Ahora cuando x1 = 6.1

Hoja terminal alcanzada: Hoja 2
Distribución de clases en la hoja: [2,0,4]
Clase predicha: Clase 2
Probabilidad empírica:

p(c=2|xb,T)=4/6=0.667

Por lo tanto, el paciente xb se clasifica como evento cardíaco adverso confirmado

Limitar la profundidad del árbol ayuda a evitar el sobreajuste, porque impide que el modelo cree divisiones demasiado específicas para los datos de entrenamiento. En problemas biomédicos, esto es importante porque los datasets suelen ser pequeños y pueden tener ruido o variabilidad entre pacientes. Un árbol muy profundo podría memorizar casos particulares en lugar de aprender patrones generales, por lo que al limitar la profundidad se mejora la capacidad del modelo para generalizar a pacientes nuevos.

Observación: Para el apartado a), considere thresholds ubicados entre valores consecutivos de cada feature ordenados de menor a mayor. No es necesario evaluar splits que generen nodos con menos de 2 muestras.